

# Client–Server Model

## Definition

The **client–server model** is an operating-system and system-design model in which one group of processes, called **servers**, provides services, while another group of processes, called **clients**, requests and uses those services.

## 1. Basic Idea

### Main Concept

In the client–server model:

- **clients** ask for services,
- **servers** perform those services,
- communication usually happens through **message passing**.

### Relation with Microkernel

The lowest layer is often a **microkernel**, but that is not required. The main idea of this model is simply the existence of client processes and server processes.

## 2. Clients and Servers

### Client

A **client** is a process that requests a service from another process.

### Server

A **server** is a process that provides a specific service to clients.

### Examples of Services

A server may provide services such as:

- file service,
- process management,
- terminal service,
- web service,
- database service.

### 3. Communication by Message Passing

#### Message Passing

Communication between clients and servers is usually done through **message passing**.

#### How It Works

To get a service:

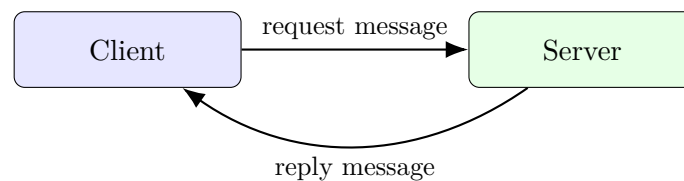
1. the client prepares a message describing what it wants,
2. the client sends the message to the server,
3. the server performs the work,
4. the server sends back a reply.

#### Key Idea

Whether the client and server are on the same machine or different machines, the basic idea is the same:

request → service → reply

#### 3.1 Simple Request–Reply Diagram



### 4. Same Machine or Different Machines

#### Local and Remote Use

The client–server model can be used:

- on a **single machine**,
- or across a **network of machines**.

#### Local Case

If the client and server are on the same machine, some optimizations are possible. However, conceptually they still communicate by message passing.

#### Remote Case

If the client and server are on different computers, the messages travel through a local-area network or a wide-area network.

## 5. Transparency of the Model

### Transparency

A major strength of the client–server model is that the client often does not need to know whether the server is:

- local on the same machine, or
- remote on another machine.

### Why This Matters

From the client’s point of view, the same basic operation happens in both cases:

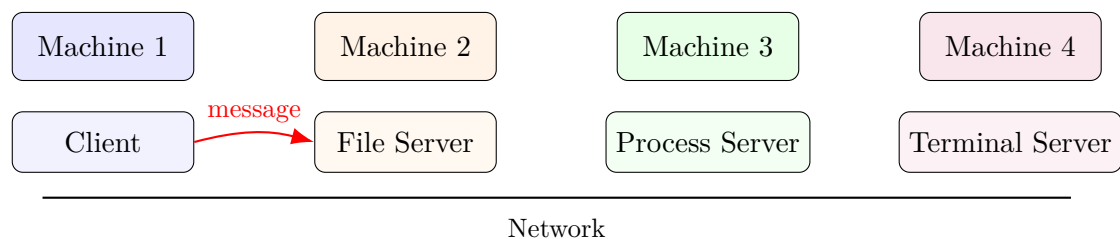
- send request,
- wait,
- receive reply.

## 6. Client–Server Over a Network

### Network Version

The client–server idea can be extended naturally to a distributed system in which clients and servers run on different computers connected by a network.

### 6.1 Network Diagram



## 7. Abstraction of the Model

### Abstraction

The client–server model is an abstraction that works in both situations:

- one computer,
- many networked computers.

So the same idea can be used for local operating-system structure or for distributed systems.

## 8. Real-Life Use

### Modern Usage

Today, many systems use home computers, laptops, or phones as clients and more powerful machines elsewhere as servers.

### Web Example

A common example is the Web:

- a PC or phone sends a request for a web page,
- the web server receives the request,
- the server sends the page back.

This is a classic example of the client–server model over a network.

## 9. Advantages of the Client–Server Model

### Advantages

The client–server model offers:

- clear separation between service provider and service user,
- modular design,
- easier distribution over networks,
- flexibility in placing services,
- easier system expansion.

## 10. Summary Points

### Quick Review

In the client–server model:

- clients request services,
- servers provide services,
- communication is usually by message passing,
- the model works on one machine or across many machines,
- the Web is a common real-world example.